

TRABAJO DE FIN DE GRADO

# Web de adopción de mascotas



## AdoptaYa

Nombre: Adrián Barreno

Curso: Desarrollo de Aplicaciones Web

Fecha: 09/05/2026

[GitHub](#)



# Contenido

|                                           |    |
|-------------------------------------------|----|
| <b>1.INTRODUCCIÓN</b>                     | 4  |
| 1.1 Objetivos                             | 5  |
| <b>2. Planificación</b>                   | 6  |
| 2.1 Metodología                           | 6  |
| 2.2 Herramienta empleada para la gestión  | 6  |
| 2.3 Temporalización                       | 7  |
| <b>3. Análisis</b>                        | 7  |
| 3.1 Estudio del mercado                   | 7  |
| 3.2 Requisitos de la aplicación           | 9  |
| 3.2.1 Requisitos funcionales              | 9  |
| 3.2.1 Requisitos no funcionales           | 11 |
| 3.3 Problemas a solucionar                | 12 |
| 3.4 Componentes del proyecto              | 13 |
| 3.5. Casos de uso                         | 15 |
| <b>4. Tecnologías</b>                     | 15 |
| 4.1 Frontend                              | 16 |
| 4.2 Backend                               | 17 |
| 4.3 Comunicación entre frontend y backend | 18 |
| 4.4 Herramientas auxiliares               | 19 |
| <b>5. Diseño</b>                          | 19 |
| 5.1 Base de datos                         | 19 |
| 5.1.1 Diagrama Entidad-Relación           | 20 |
| 5.1.2 Decisiones de diseño que tomé       | 20 |
| 5.2 Interfaz gráfica                      | 20 |
| <b>6. Implementación</b>                  | 30 |
| 6.1 Tecnologías empleadas                 | 30 |
| 6.2 Desarrollo de la aplicación (fases)   | 30 |
| 6.3 Despliegue                            | 32 |
| <b>7. Pruebas</b>                         | 33 |
| 7.1. Pruebas funcionales                  | 33 |
| 7.2. Pruebas no funcionales               | 35 |
| 7.3. Pruebas de seguridad                 | 36 |
| <b>8. Conclusiones</b>                    | 36 |
| 8.1. Problemas encontrados                | 36 |
| 8.2. Futuras mejoras                      | 37 |

|                             |           |
|-----------------------------|-----------|
| 8.3. Resumen final.....     | 37        |
| <b>9. Bibliografía.....</b> | <b>38</b> |

# 1.INTRODUCCIÓN

Cada año, miles de animales son abandonados. Muchos acaban en refugios y protectoras las cuales tienen pocos recursos para seguir adelante e intentar encontrarles una nueva familia. En este momento, el proceso para adoptar una mascota es poco eficiente: las protectoras publican los animales en redes sociales, anuncios sueltos o webs propias, y las personas que quieren adoptar a un animal tienen que buscar en muchos sitios sin un único punto de referencia.

## Motivación

La idea de este proyecto surge de un interés personal por el mundo animal y por la posibilidad de aplicar lo aprendido durante el curso algo que tuviera uso real. Quería que mi proyecto no fuera únicamente un trabajo del curso, sino una página web que pudiera existir y aportar algo en este mundillo.

Además, este proyecto me hacía desarrollar una página web completa y funcional de tal manera que hemos visto en el curso con backend, frontend, base de datos y la comunicación entre ellos.

Para más seguridad, implementé un sistema de seguridad para la parte de los refugios. Al registrarse como refugio, aparece un nuevo campo obligatorio para rellenar, que es el CIF y al crear la cuenta queda en pendiente y hasta que el admin desde el panel de administrador no se desbloquean las funciones de refugio para ese usuario.

Otra característica que tiene el proyecto es el sistema de adopción. Lo quise hacer lo más realista posible: cuando un adoptante solicita la adopción a una mascota, le aparece un formulario donde tendrá que detallar la información sobre su vivienda, experiencia con mascotas, su situación laboral, etc. La solicitud pasa por varios estados: pendiente, en revisión y aceptado o rechazado. Esto lo gestiona el refugio, cuando se acepta esto, la mascota queda reservada hasta que el refugio confirme manualmente la entrega de esta mascota. Esto ayuda a gestionar el sistema de adopción y lo hace más realista y profesional.

## Descripción del proyecto

El proyecto, llamado AdoptaYa, es una aplicación pensada para facilitar el proceso a personas que quieran adoptar a una mascota. Una de las funciones principales es que las protectoras y los adoptantes tienen una página de mensajería directamente para poder hablar entre ellos. La idea es centralizar en un sitio lo que está repartido por muchos, es decir, juntas todas las protectoras y poder ver todas las mascotas en un sitio en vez de ir sitio por sitio.

Dentro de la aplicación conviven tres tipos de usuario:

- Las protectoras, crean su perfil y publican las mascotas disponibles.
- Los adoptantes, pueden ver todas las mascotas, filtrar por lo que quiera buscar, y solicitar adoptar a la mascota que les ha gustado.
- Un administrador, supervisa el funcionamiento general desde un panel de control.

## 1.1 Objetivos

### 1.1.1 Objetivo general

El objetivo de este proyecto es desarrollar una aplicación web completa que facilite el proceso de adopción de mascotas, conectando a los refugios y protectoras con las familias o personas que quieran adoptar una mascota. La aplicación está pensada para que una gestión, que normalmente son llamadas, búsqueda en redes sociales y en muchas páginas, se centre en una sola página con toda la gestión y abierto para todo el mundo.

### 1.1.2 Objetivos específicos

A partir del objetivo general, se me pasaron por la cabeza otros objetivos más concretos que quiero llevar a cabo en el desarrollo de este proyecto.

1. **Construir una aplicación web con arquitectura real.** En este proyecto uno de los objetivos más concretos es saber aplicar en él los conocimientos que he adquirido tanto en este curso como en el anterior, realizando tanto el frontend como el backend y que se comuniquen entre ambas a través de API REST para hacer la aplicación mucho más completa.
2. **Implementar un sistema de roles.** Para que la página sea más realista y funcional, decidí que uno de los objetivos sea añadir un sistema de roles, donde tengo tres tipos diferente de roles que se le asignan a cada usuario. Estos roles son adoptantes, refugio o protectora, y administrador.
3. **Implementar un sistema de mensajería:** tenemos un sistema de mensajería donde refugios y adoptantes pueden estar en contacto sin necesidad de salirse de la página web.
4. **Garantizar la seguridad de datos personales de los usuarios.** En este tipo de aplicaciones donde los usuarios se registran con sus datos y donde ponen una contraseña, hay que tener muy buena seguridad para la protección de datos y que nadie externo pueda obtener los datos de los usuarios. Para ello tenemos estas medidas de seguridad: hash de contraseñas con bcrypt, autenticación mediante tokens JWT, validación de roles en cada petición sensible y control de acceso.
5. **Permitir la gestión de usuarios, mascotas y solicitudes a través de un panel de administración.** El usuario administrador tiene una función diferente a los demás y es la del panel de administración donde puede supervisar el funcionamiento de la página, gestionar los usuarios y roles, moderar las mascotas publicadas y revisar las solicitudes de adopción. Este panel hace la aplicación auto-gestionable.
6. **Aprender nuevas tecnologías y lenguajes de programación.** Este objetivo más que técnico es un objetivo personal que tengo conmigo mismo. Se centra en absorber toda la información y nuevas enseñanzas que pueda para poder desarrollar por primera vez una aplicación web completa con su backend, frontend y la unión entre ambas.
7. **Implementar un sistema de verificación para refugios:** asegurar de que solo los refugios reales pueden registrarse y hacer todo lo que puede hacer un refugio en la aplicación.
8. **Implementar un proceso de adopción realista:** que, al solicitar una adopción, la solicitud incluya un formulario con la información del adoptante, pase por una gestión por el refugio, y que siempre quede reflejado el estado de esa mascota.

## 2. Planificación

### 2.1 Metodología

Para hacer bien el proyecto he decidido ir por una metodología por fases, he dividido el trabajo por partes teniendo sentido el orden de realización y que se puedan ir enlazando parte por parte terminando una a una. La intención era no liarme al hacerlo todo a la vez e ir poco a poco y haciendo siempre algo efectivo y teniendo clara la idea.

Las fases que seguí, más o menos en este orden fueron las siguientes:

1. **Fase de planteamiento:** pensar que iba a hacer la aplicación, que tecnologías iba a utilizar y cómo iba a estructurar la base de datos.
2. **Fase de backend:** montar el servidor con Node y Express, conectar con MySQL y dejar funcionando los endpoints principales como por ejemplo el login, el registro y el listado de las mascotas.
3. **Fase de frontend:** empezar a construir la interfaz con React, conectarla con la API del backend e ir creando todas las páginas necesarias.
4. **Fase de pruebas y memoria:** probar absolutamente toda la aplicación para que todo funcione y redactar la memoria.

Aunque lo planteé con fases ordenadas, alguna se ha ido juntando o he necesitado hacer cosas de otra fase para poder seguir con el proyecto.

### 2.2 Herramienta empleada para la gestión

Para la gestión del proyecto he utilizado GitHub principalmente. A parte de servirme como repositorio del código, me ha servido para:

1. Tener control de versiones del proyecto, de manera que cualquier cambio queda registrado y se puede volver a atrás por si me da algún problema más adelante.
2. Ver el historial de que voy haciendo a través de los commits que hago, lo cual me ha servido también como “agenda” de las cosas que hacía y cuando.
3. No me preocupaba el tema de poder perder el código que tenía ya bien hecho porque lo tenía guardado en la nube y podía tenerlo en cualquier dispositivo sin la necesidad de ir pasándoselo cada dos por tres.

GitHub no solo lo he utilizado para guardar código sino como una herramienta que me ha ayudado a tener orden con mi proyecto.

## 2.3 Temporalización

El proyecto se ha desarrollado entre el mes de febrero y el 4 de mayo ya que el 4 de mayo era la entrega para la revisión para dar los últimos retoques. La entrega definitiva es el 11 de mayo. Al final en total me ha costado unas 50 horas realizar el proyecto, y están repartidas más o menos como muestro ahora:

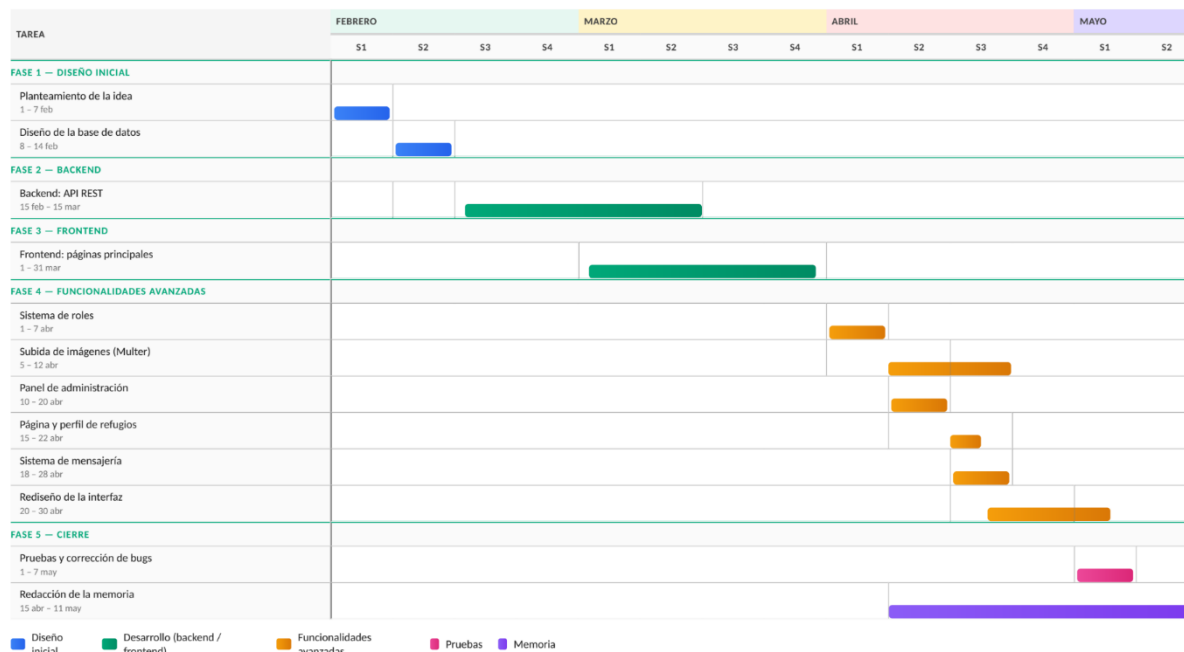


Ilustración 1 Diagrama de Gantt

Las primeras fases y meses las dediqué al desarrollo, empecé por el backend y luego poco a poco iba metiendo frontend, y con el tiempo el proyecto iba cogiendo forma de un proyecto real. Luego iba metiendo las funciones, los diferentes roles y la parte de la administración. La parte final la he centrado en los últimos retoques de la página y en desarrollar la memoria del proyecto con toda la información de cómo y qué he hecho.

## 3. Análisis

### 3.1 Estudio del mercado

Para realizar la aplicación antes de empezar a desarrollar y estructurar la base de datos me fijé en diferentes páginas web para ver qué funciones puedo implementar en mi proyecto. La intención era buscar que es lo más viable para realizar un proyecto real como el mio.

Me fijé en estas 3 páginas web que son las más conocidas en este sector:

#### Yaencontre Mascotas

Esta página es de las más conocidas en España en el tema de adopciones de mascotas. Ofrece un montón de mascotas para poder adoptar con un buscador y un filtro de animales que se filtran por tipo, edad, ciudad, etc.

| Puntos fuertes                                                   | Puntos débiles                                                          |
|------------------------------------------------------------------|-------------------------------------------------------------------------|
| La variedad de mascotas y de refugios es muy amplia.             | La interfaz la veo muy cargada y es poco moderna.                       |
| Los filtros son muy completos y se llega a donde quieres llegar. | Mezcla las mascotas con anuncios de venta.                              |
| Hay refugios y mascotas de toda España.                          | No tiene un sistema muy claro de cuenta para los refugios y protectoras |

## Wakyma

Esta es super conocida en este mundo de las adopciones, la conocen como usuarios o familias que quieren adoptar, como las protectoras. Es parecido a un tablón de anuncios y cada uno publica sus mascotas.

| Puntos fuertes                           | Puntos débiles                                                     |
|------------------------------------------|--------------------------------------------------------------------|
| Es totalmente gratis.                    | Todos los usuarios pueden publicar mascotas, no solo los refugios. |
| Tiene un montón de animales disponibles. | El diseño de la página es bastante antiguo.                        |
|                                          | No diferencian nada entre protectoras o refugios, y particulares.  |

## Barkibu

Esta aplicación web es bastante más moderna, está centrada sobre todo en la parte de veterinaria y cuidado animal, pero tiene una parte de adopciones que es en la que yo me fijé. Se diferencia a las demás porque está diseñada más para utilizarla en el móvil y tiene un enfoque mucho más visual.

| Puntos fuertes                                                                    | Puntos débiles                                                       |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Tiene un diseño muy moderno y bastante limpio.                                    | El punto principal no es la adopción y mi página solo va sobre eso.  |
| Se adapta muy bien a cualquier dispositivo                                        | Tienen pocos animales disponibles.                                   |
| No solo se centra en las adopciones, también en cosas que entran en ese mundillo. | No está pensada mucho para usuarios nuevos, más para dueños actuales |



## Conclusión del estudio

Al analizar estas páginas me di cuenta de que hay muchas páginas web de adopciones, pero las que hay o son antiguas y poco modernas o son muy poco intuitivas para los usuarios nuevos. Y las que son modernas e intuitivas no se enfocan a las adopciones, sino que tienen una función principal, como por ejemplo el cuidado animal y veterinaria. No hay una página web moderna e intuitiva que únicamente se centre en conectar los refugios con los usuarios y familias que quieren adoptar a una mascota. Y con un sistema de roles claro y diferenciado por sus diferentes funciones. Y ese es el hueco que quiero cubrir con mi proyecto.

Otra cosa en la que me he fijado es la fiabilidad que tienen las páginas web con los refugios. En alguna página puede registrarse como refugio cualquiera y poder añadir mascotas, sin tener ningún tipo de verificación, y de esa forma para los adoptantes es poco fiable.

## 3.2 Requisitos de la aplicación

A la hora de empezar a hacer el proyecto, separé los requisitos en dos tipos: los funcionales, que dicen que debe hacer la página, y los no funcionales, que describen cómo se hace todo.

### 3.2.1 Requisitos funcionales

Estas son las funciones que la aplicación hace y yo las he dividido por los diferentes tipos de usuarios y roles que hay.

#### Visitante

| Código | Requisito                                                      |
|--------|----------------------------------------------------------------|
| RF-01  | Puede ver la página principal con toda la información general. |
| RF-02  | Puede ver todo el catálogo de mascotas.                        |
| RF-03  | Puede filtrarlas mascotas por tipo, ciudad y nombre.           |
| RF-04  | Puede ver todos los detalles de cada mascota.                  |
| RF-05  | Puede ver todos los refugios que colaboran.                    |
| RF-06  | Puede registrarse como un usuario(adoptante) o como refugio.   |
| RF-07  | Puede iniciar sesión con su cuenta.                            |

#### Adoptante

| Código | Requisito                     |
|--------|-------------------------------|
| RF-08  | Puede ver todos los animales. |

|       |                                                                                               |
|-------|-----------------------------------------------------------------------------------------------|
| RF-09 | Puede mandar una solicitud de adopción a un refugio.                                          |
| RF-10 | Puede comunicarse con los refugios.                                                           |
| RF-11 | El adoptante tiene que rellenar un formulario con datos personales al solicitar una adopción. |
| RF-12 | Podrá ver todas las solicitudes que ha mandado y el estado de cada una.                       |

### Refugio (protectora)

| Código | Requisito                                                                                      |
|--------|------------------------------------------------------------------------------------------------|
| RF-12  | Puede crear, editar y subir una imagen de su perfil que es público.                            |
| RF-13  | Puede publicar nuevos animales al catálogo de mascotas.                                        |
| RF-14  | Puede subir una imagen para las mascotas.                                                      |
| RF-15  | Pueden comunicarse con los adoptantes.                                                         |
| RF-16  | Puede aceptar las solicitudes de adopción                                                      |
| RF-17  | Debe poner su CIF en el registro                                                               |
| RF-18  | No podrá realizar las funciones de un refugio hasta que el administrador verifique su cuenta.  |
| RF-19  | El refugio puede ver una lista de sus propias mascotas publicadas                              |
| RF-20  | Puede aceptar o rechazar las solicitudes de adopción                                           |
| RF-21  | Podrá ver las solicitudes de adopción de sus propias mascotas                                  |
| RF-22  | Podrá aceptar o rechazar las solicitudes de adopción de sus propias mascotas.                  |
| RF-23  | Cuando acepte la solicitud, la mascota será adoptada.                                          |
| RF-24  | Puede ver los datos personales del adoptante, del formulario de la solicitud                   |
| RF-25  | Puede marcar como revisión una solicitud para saber que la están revisando.                    |
| RF-26  | Al aprobar una solicitud, la mascota pasa a estar en “reserva”                                 |
| RF-27  | Puede marcar una mascota que está reservada como adoptada cuando la mascota se haya entregado. |

## Administrador

| Código | Requisito                                                       |
|--------|-----------------------------------------------------------------|
| RF-28  | Puede cambiar el rol de todos los usuarios                      |
| RF-29  | Puede eliminar usuarios                                         |
| RF-30  | Puede listar todas las mascotas                                 |
| RF-31  | Puede eliminar las mascotas                                     |
| RF-32  | Puede ver las solicitudes de adopción.                          |
| RF-33  | Puede aceptar o rechazar solicitudes                            |
| RF-34  | Al aceptar la solicitud, la mascota se marca como adoptada      |
| RF-35  | Puede listar todos los usuarios del sistema.                    |
| RF-36  | Podrá ver el CIF de cada refugio y su estado de verificación    |
| RF-37  | Podrá verificar, desde el panel de administración, a un refugio |

### 3.2.1 Requisitos no funcionales

| Código | Requisito                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------|
| RNF-01 | Seguridad: las contraseñas están guardadas hasheadas con bcrypt, no se guardan nunca en texto plano.                        |
| RNF-02 | Autenticación: la sesión se guarda con tokens JWT con caducidad de 7 días                                                   |
| RNF-03 | Validación de archivos: se pueden subir imágenes JPG, JPEG, PNG Y WEBP pero con un máximo de 5MB.                           |
| RNF-04 | Validación de roles: cada acción única de un rol solo aparecerá para dicho rol.                                             |
| RNF-05 | Diseño responsive: la página web se podrá ver bien en cualquier dispositivo.                                                |
| RNF-06 | Usabilidad: la interfaz es clara y muy intuitiva para nuevos usuarios.                                                      |
| RNF-07 | Rendimiento: las páginas se cargan en menos de 2 segundos.                                                                  |
| RNF-08 | Escalabilidad: el código está organizado para tener escalabilidad a futuros                                                 |
| RNF-09 | Control de versiones: utiliza GitHub para llevar el historial de cambios                                                    |
| RNF-10 | Privacidad de los mensajes: solo el emisor y el receptor pueden ver los mensajes y toda la conversación.                    |
| RNF-11 | Fiabilidad de las publicaciones: Los refugios únicamente pueden añadir mascotas cuando sea verificado por el administrador. |
| RNF-12 | El sistema impide solicitar una adopción a un usuario, a las mascotas que ya ha mandado una solicitud                       |

### 3.3 Problemas a solucionar

En la introducción ya he comentado que hoy en día adoptar a una mascota es más difícil de lo que parece. Ahora vamos a entrar al detalle en los problemas que quiero o pretendo resolver con mi proyecto.

#### 1. Información dispersa

- a. Problema: Cada protectora pública sus mascotas por su cuenta, las suelen subir a Instagram, Facebook, su propia web, anuncios... Las personas que quieren adoptar

para llegar a lo que quieren igual tienen que buscar en muchas páginas, anuncios, publicaciones diferentes.

- b. Solución de AdoptaYa: nosotros centralizamos las protectoras y las mascotas. Todo aquello que antes lo veías en diferentes sitios ahora lo puedes ver en un único sitio.

## **2. Falta de filtros eficientes**

- a. Problema: Las redes sociales y los anuncios no sirven para buscar directamente lo que quieres encontrar. Tienes que estar bastante tiempo buscando, haciendo scroll y eso a veces es una situación muy frustrante.
- b. Solución: tenemos unos filtros donde puedes buscar directamente lo que quieres y así te ahorras tiempo y es mucho más cómodo que estar buscando por todos lados.

## **3. Comunicación dispersa entre adoptante y refugio**

- a. Problema: cuando una persona o familia quiere adoptar a una mascota, siempre tiene dudas, de su tamaño, comportamiento, si se lleva bien con las personas, etc. Y esto a veces es difícil llegar a preguntar o te contestan tarde porque el sistema de mensajes no es el mejor.
- b. Solución: Incluimos un sistema de mensajería dentro de la aplicación donde puedes contactar con un usuario de la forma más cómoda y sin salirte de la aplicación.

## **4. Cada protectora gestiona como puede**

- a. Problema: las protectoras como herramienta de gestión utilizan cosas manuales como excel, libretas, mensajes de WhatsApp... Y no tienen herramientas pensadas para esto.
- b. Solución: le damos a las protectoras un perfil único y propio donde pueden crear su perfil y tienen la posibilidad de subir las mascotas con la información, gestionar sus datos y también salen públicamente en la página web.

## **5. Dificultad para el seguimiento de las solicitudes**

- a. Problema: cuando alguien manda un mensaje o llama a una protectora por un animal no queda registrado en ningún sitio esa solicitud. es fácil que se pierda esa información.
- b. Solución: nosotros esta información la dejamos registrada en la base de datos, sabemos quién hace la solicitud y a qué refugio se la hace.

# **3.4 Componentes del proyecto**

Este proyecto lo forman tres diferentes partes que son las más importantes, son las que juntas hacen que funcione la aplicación, pero que claramente están separadas. Esta separación se puede presenciar claramente en el código.

## **Frontend**

Es la parte visual del proyecto, es la que el usuario ve al abrir la página en el navegador. Está hecha con React y está conectada con el servidor a través de peticiones HTTP. Se encarga de:

- Muestra todas las páginas diferentes.
- Recoge los datos de los usuarios de los formularios.
- Envía los datos al backend y muestra la respuesta.
- Gestiona la sesión de los usuarios en el navegador.

## Backend

Esta es la parte que el usuario no puede ver, pero es el que gestiona todo y el que hace que todo funcione. Está hecho con [Node.js](#) y express, y expone una API REST a la que el frontend manda peticiones. Se encarga de:

- Recibe peticiones del frontend.
- Valida datos y los permisos que tiene cada usuario.
- Se comunica con la base de datos.
- Devuelve al frontend en forma de JSON la información.
- Gestiona la subida de imágenes.

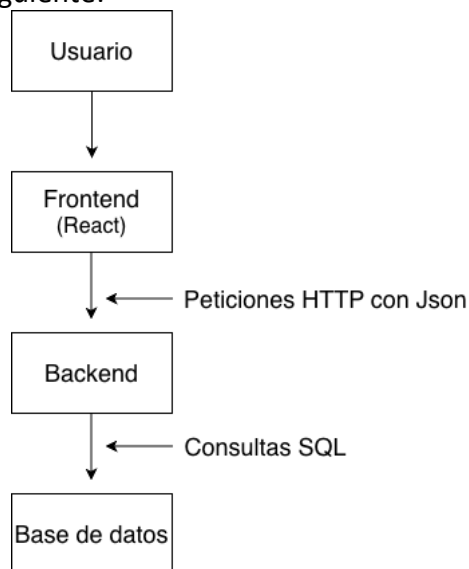
## Base de datos

Aquí es donde se almacena toda la información. Está hecha con MySQL y contiene 5 tablas principales:

- Usuarios: aquí están todas las cuentas registradas.
- Mascotas: aquí se almacenan las mascotas disponibles para adoptar.
- Solicitudes\_adopcion: aquí están las solicitudes de los adoptantes.
- Refugios: todos los refugios que colaboran para poder adoptar sus mascotas.
- Mensajes: toda la información del sistema de mensajería

## Comunicación entre ellos

El esquema general es el siguiente:



Cuando el usuario interactúa en la página, el frontend manda peticiones al backend, el backend la procesa, consulta o modifica la base de datos si hace falta, y devuelve la respuesta al frontend para que pueda verla el usuario.

Esta arquitectura (cliente-servidor) es la más común en las aplicaciones modernas. Tiene una ventaja y es que cada parte puede modificarse independientemente: por ejemplo, si el día de mañana esta web la quieres pasar a una aplicación móvil no tienes que tocar el backend, solo únicamente el frontend.

### 3.5. Casos de uso

Los casos de uso describen las interacciones y funciones más importantes que se pueden hacer en la página web. He organizado los casos de uso por los diferentes tipos de usuarios que hay.

#### Tipos de usuarios

- Visitante: es el usuario sin iniciar sesión, solo puede ver la información pública. El catálogo de mascotas y la lista de refugios que se registran y crean un perfil.
- Adoptante: tiene todas las funciones de un adoptante, puede comunicarse con el refugio y solicitar una adopción.
- Refugio: puede añadir una mascota al catálogo, crearse un perfil de su propio refugio y se añade a la lista de refugios, y también gestiona las solicitudes de sus mascotas.
- Administrador: tiene permisos de control total. Puede gestionar usuarios, refugios, mascotas y solicitudes.

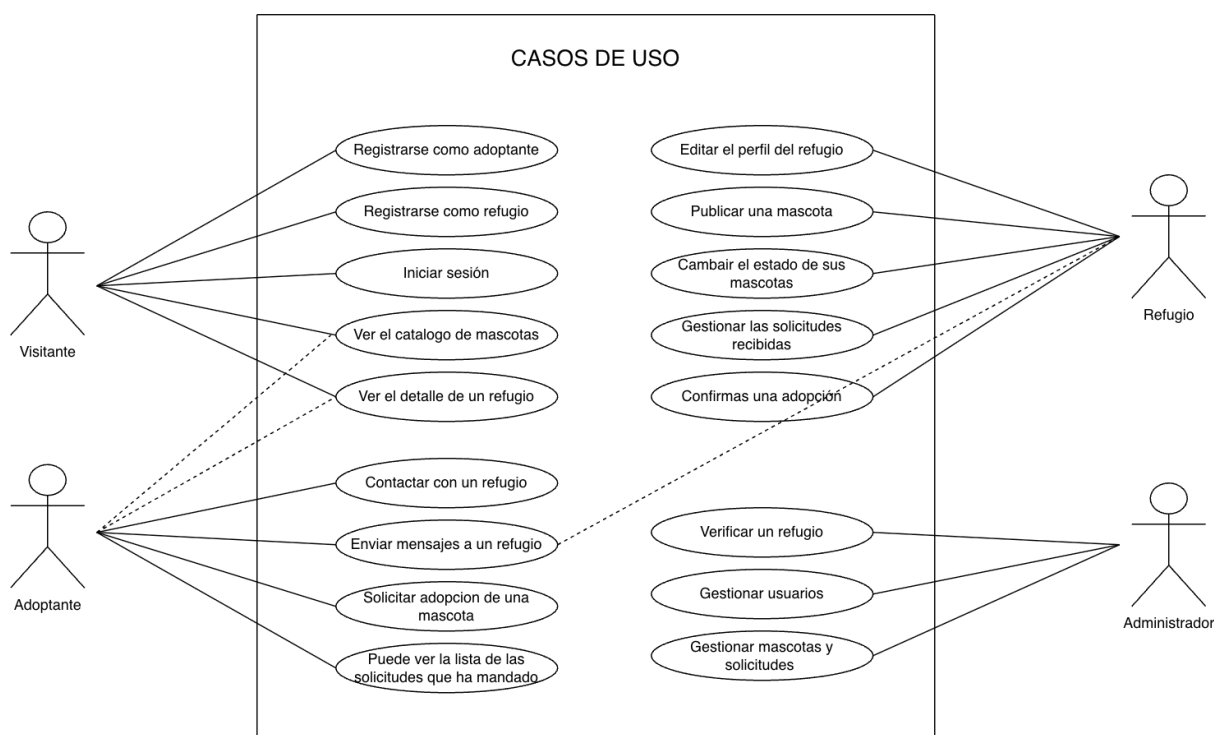


Ilustración 2 Casos de Uso

## 4. Tecnologías

En este apartado describo todas las tecnologías que he utilizado para este proyecto. Los he dividido en tres partes diferentes, en el frontend (Lo que el usuario ve y con lo que interactúa en el navegador), el backend (La parte del servidor) y las herramientas auxiliares que han ayudado para el proceso de desarrollo del proyecto.

Estas tecnologías las he elegido porque actualmente son las que más se utilizan y se pueden conectar bien entre sí. Esto me ha ayudado a aprender a trabajar con un proyecto real.

## 4.1 Frontend

### React

React es una librería de JavaScript, creada por Facebook pero ahora mismo pertenece a Meta, para construir las interfaces de usuarios. Es una de las tecnologías de frontend más utilizadas en el desarrollo web actual junto a Angular y Vue.

La idea de React es dividir la interfaz en componentes: son piezas pequeñas que más adelante pueden ser reutilizables como, por ejemplo: un botón, una tarjeta, un formulario. Estos componentes se combinan para formar páginas completas. En mi proyecto hay componentes como un Navbar, un Footer, etc, y páginas que los utilizan como Home, Adoptar o Refugios. He elegido esta tecnología porque:

- Es una de las librerías que más piden saber utilizar en ofertas de trabajo.
- Tiene tanta comunidad que si tienes dudas sobre algo esta duda está resuelta en algún sitio.
- Permite separar bastante bien la lógica de la presentación, por eso es mucho más fácil mantener el código.

### Vite

Esta herramienta es la que compila el código de React (exactamente los archivos .jsx) a JavaScript que el navegador puede entender. Levanta un servidor de desarrollo que cada vez que guardo un cambio se actualiza automáticamente, esto hace que el desarrollo sea muy ágil.

Es una alternativa moderna y más rápida que las herramientas más antiguas como Webpack. En este proyecto arranco el frontend bastantes veces al día, esa diferencia de velocidad se nota bastante.

### CSS plano

Para el diseño de la aplicación web opté por un CSS plano, al principio hice el diseño con Tailwind donde tenía cosas pre-hechas donde es más fácil diseñar sin pensar tanto. Pero al terminarlo vi que muchas páginas web se parecían a la mía, entonces decidí migrar el diseño de la página a CSS para tener un diseño mucho más personalizado y personal.

He aprovechado funcionalidades de CSS como:

- Variables como `—acento`, `—gris-100`, etc. que lo que hace es mantener una paleta de colores coherente y fácil de cambiar.
- Grid y flexbox para el sistema de las tarjetas o rejillas y la disposición de los elementos.
- Media queries hace que la página sea responsive.
- Transiciones y animaciones para que la página se vea más moderna y no se vea tan estática.

### IntersectionObserver

Es una API del navegador, esto no es una librería externa, permiten detectar cuando un elemento que se puede ver por el usuario. La uso para ayudarme a activar las animaciones de



aparición cuando el usuario hace scroll: las tarjetas, los features... aparecen suavemente cuando el usuario haciendo scroll llega a esa parte de la página. Frente a otras opciones como librerías tipo AOS o GSAP, esta opción no añade peso a la página y es más eficiente porque las librerías tienen que estar escuchando al evento scroll todo el rato y esta API no.

## 4.2 Backend

### Node.js

Es un entorno donde se puede ejecutar JavaScript en el servidor. Antes cuando Node no existía, Javascript solo se podía utilizar en la parte de frontend, en lo que el usuario podía ver en el navegador, con Node se puede usar también para hacer servidores, conectarse a bases de datos, etc.

He elegido Node por estas razones:

- Te deja utilizar el mismo lenguaje para backend y frontend.
- Para aplicaciones web que realizan muchas peticiones a la vez es más efectivo y muy rápido.
- Tienen un montón de librerías (npm) que cubren todas las necesidades.

### Express

Es un framework para Node.js que lo que hace es simplificar mucho la creación de servidores web. Sin Express, hacer un servidor utilizando Node es bastante engorroso. Con Express se hace todo mucho más directo y te deja organizar las rutas y los controladores de forma limpia.

He utilizado express por las siguientes razones:

- Recibir las peticiones HTTP que llegan del frontend.
- Poder ejecutar los middlewares.
- Llamar a los controladores gracias a la URL y el método HTTP.
- Devuelve las respuestas en un JSON.

### MySQL

Es el sistema de gestión de mi base de datos. Lo he elegido porque es lo que hemos visto en clase y es el más factible para mi proyecto ya que es estable y gratuito.

La base de datos de mi proyecto tiene 5 tablas principales: usuarios, refugios, mascotas, solicitudes\_adopcion y mensajes. Están conectadas entre sí mediante claves foráneas.

Como cliente para gestionar mi base de datos yo he usado phpMyAdmin, que viene con Amps y permite utilizar MySQL de forma visual en el navegador.

### JWT (JSON Web Tokens)

Con JWT yo he gestionado la autenticación. Esto funciona así: un usuario inicia sesión correctamente, el servidor entrega al usuario un "token", que es una cadena cifrada, y el navegador la guarda. Después de eso cada vez que ese usuario hace peticiones que requiere

estar logueado, el navegador manda su token en la cabecera HTTP, y el servidor verifica si es real.

Las ventajas de utilizar JWT:

- No te hace falta guardar el estado de las sesiones en el servidor, lo que hace que la arquitectura se simplifique.
- El token contiene los datos del usuario cifrados, eso te hace que no tengas que llamar a la base de datos en cada petición.
- Es la opción más usada en APIs REST modernas.

### **bcryptjs**

Para tener más seguridad en mi proyecto nunca guarde las contraseñas en texto plano. Utilizo bcryptjs que es una librería diseñada para hashear las contraseñas.

El proceso es:

1. Cuando el usuario crea su usuario, su contraseña se procesa y se guarda en la base de datos en forma de hash.
2. Cuando se intenta iniciar sesión se vuelve a procesar esa contraseña y se compara el hash con el que está guardado.

Lo más importante es que el hasheo de bcryptjs no se puede revertir a la contraseña: aunque alguien tenga acceso a la base de datos no va a tener acceso a la contraseña.

### **Multer**

Con multer gestiono la subida de archivos, ya que es una librería para eso. Con esta librería lo que hago posible es la subida de una imagen en el perfil de un refugio y que los refugios puedan añadir una mascota con su imagen.

Multer recibe los archivos que manda el usuario, los guarda en una carpeta en el servidor con un nombre único que se basa en la fecha actual y nos da el nombre final para poder guardarlo en la base de datos.

### **CORS**

Los navegadores no permiten que una página web pueda hacer peticiones a un servidor que está en un dominio o puerto distinto.

CORS es el mecanismo que permite indicar al navegador que el servidor acepta peticiones desde otros sitios

## **4.3 Comunicación entre frontend y backend**

### **API REST**

La comunicación entre frontend y backend se hace con una API REST. Esto significa:

- Cada recurso tiene su propia URL.
- Las acciones se hacen con los métodos HTTP estándar: GET para consultar, POST para crear, PUT para modificar y DELETE para borrar.
- Los cambios de los datos se hacen en formato JSON.
- El stateless: el servidor no guarda información sobre la conversación previa, así que cada petición debe tener todo lo necesario.

## **Polling**

He utilizado polling para el sistema de mensajería entre adoptantes y refugiados. Polling es una técnica que cada cierto tiempo, el frontend mira en el backend si hay mensajes nuevos, y según si hay o no se actualiza la conversación.

Existe una alternativa al polling que es el WebSockets, que suele ser lo que utilizan aplicaciones conocidas con WhatsApp para la comunicación rápida e instantánea, pero es una técnica mucho más compleja y difícil de hacer.

## **4.4 Herramientas auxiliares**

### **Visual Studio Code**

Este es el editor de código que he utilizado para el desarrollo de mi proyecto. Es gratuito, ligero y tiene un montón de extensiones disponibles y esto hace que este editor de código sea la opción más práctica para cualquier lenguaje.

### **Git y GitHub**

Git es el sistema de control de versiones que he estado utilizando para llevar el historial de lo que voy haciendo y los cambios que hago. En cada cambio hago un commit, que es lo que me permite volver atrás si tengo problemas y se me rompe.

GitHub es la plataforma donde tengo el repositorio subido. Me sirve como copia de seguridad que está en la nube. También me ha servido como agenda del proyecto donde mirando los commits sé lo que llevaba hecho.

En este caso iba trabajando con diferentes ramas para asegurarme de que mi proyecto va a estar a salvo de perderse.

## **5. Diseño**

Después de explicar y las tecnologías que uso y los requisitos que pretendo cumplir con la aplicación, ahora voy a explicar el diseño. Como tengo estructurada la base de datos y cómo iba a verse la aplicación de cara al usuario.

### **5.1 Base de datos**

El diseño de la base de datos la he pensado de tal manera que sea lo más sencillo posible hacerla sin perder ningún dato importante. He evitado tablas innecesarias o demasiado complejas y tener bien definidas las relaciones.

La base de datos está formada por cinco tablas principales que se relacionan entre ellas a través de claves foráneas.

### 5.1.1 Diagrama Entidad-Relación

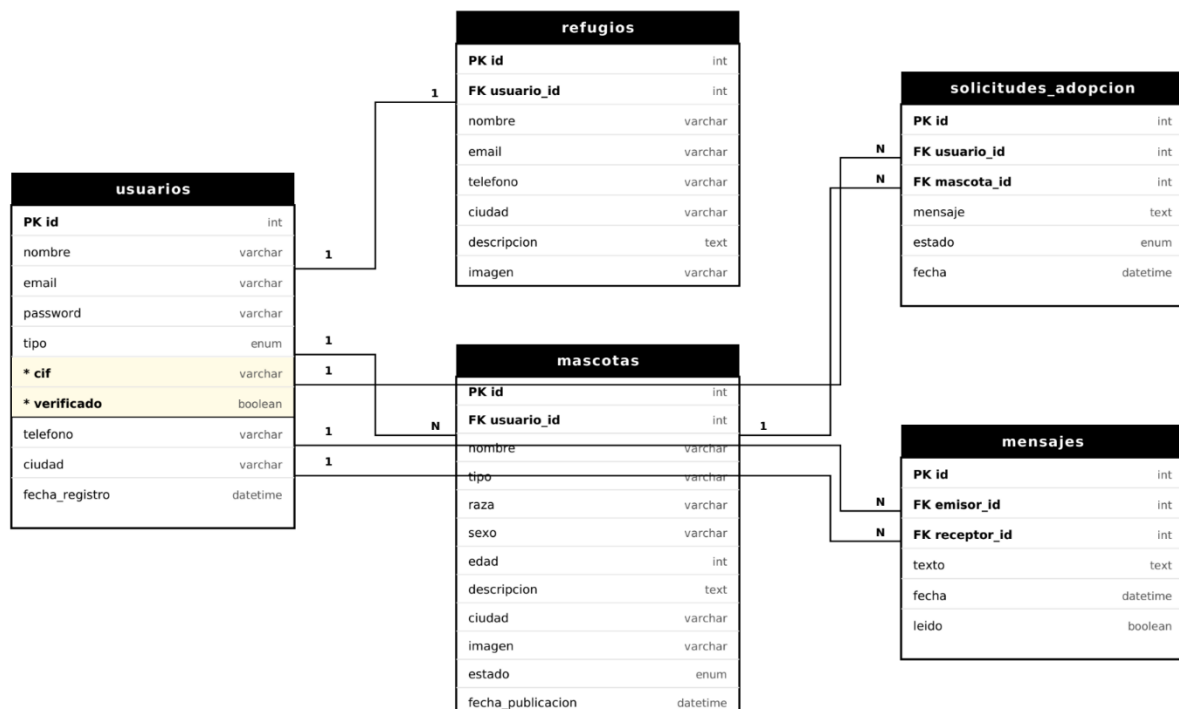


Ilustración 3 Diagrama de Entidad-Relación

Como se puede ver en la imagen, en mi diagrama de entidad-relación todas las tablas giran en torno de la tabla usuarios, ya que es el centro de todo. Y eso tiene sentido ya que tanto refugios, como adoptantes y administradores son usuarios.

Y por otra parte tengo la tabla de refugios por el tema del perfil del refugio, etc. Así puedo guardarlos y mostrarlos en una página independiente.

### 5.1.2 Decisiones de diseño que tomé

Mientras hacía la estructura de la base de datos tuve que tomar unas decisiones:

- Solo hacer una única tabla de usuarios para los tres roles diferentes: en vez de tener dos tablas una con los usuarios y otra con los roles, decidí tener solo una tabla de usuarios con el campo tipo. Así puedo simplificar las consultas y también permite que la lógica de autenticación sea común para todos.
- Tabla refugio separada: tener una tabla de refugios a parte permite que los campos sean opcionales y que solo la rellenen quien les interese o quien sea un refugio.
- Claves foráneas con borrado en cascada: Cuando se borra un usuario toda la información que esté relacionada con ese usuario se elimina automáticamente.
- Las contraseñas se guardan hasheadas: el campo password no tiene la contraseña real, sino tiene un hash que hace tener más seguridad ya que se ve cifrada.
- Verificación en la tabla de usuarios: El campo de verificación la añadí directamente en la tabla de usuarios, en vez de crear una tabla nueva para la verificación. Porque es un dato bastante simple (true/false).

## 5.2 Interfaz gráfica

El diseño de la interfaz es lo que más tiempo he empleado. Mi objetivo era tener un diseño moderno, claro e intuitivo, sin tener un diseño que parece hecho por plantilla genéricas.

## Identidad visual

Antes de diseñar las páginas pensé como quería estructurar la aplicación web para saber cómo empezar a diseñar.

## Paleta de colores

He optado por unos colores principales para una paleta minimalista.

| Color           | Hex     | Uso                                                   |
|-----------------|---------|-------------------------------------------------------|
| Negro           | #0A0A0A | Texto principal, botones primarios.                   |
| Blanco          | #FFFFFF | Fondo y el contraste                                  |
| Verde esmeralda | #00A878 | Color de acento, interacciones y elementos destacados |

Principalmente he utilizado esta paleta de colores, pero también he añadido una escala de grises para los textos secundarios, los bordes y para los fondos suaves. La idea era que el verde fuera para cosas importantes, de manera que guíe al usuario y destaque.

## Tipografía

La fuente que he elegido es Inter, es moderna y está diseñada específicamente para interfaces digitales. Esta fuente se carga directamente desde google fonts.

He elegido Inter porque:

- Es muy legible en pantalla, incluso con tamaño pequeño.
- Es polivalente con los idiomas, soporta palabras con tildes y “ñ”.
- Es la tipología que utilizan webs como GitHub o Vercel.

## Reorganización del menú de usuario

Al principio, todos los enlaces los tenía en el navbar, lo que llegaba a una sensación de saturación en el navbar. Para que no generase esa sensación, organicé el navbar, en un menú desplegable. Las acciones principales, las he mantenido en el navbar, pero las acciones de cuenta como, por ejemplo: mi refugio, mis mascotas, etc, están en el menú desplegable. Así me aseguro de que al entrar en la página no se vea saturada y únicamente se vean los enlaces principales.



Ilustración 4 Menú de adoptante

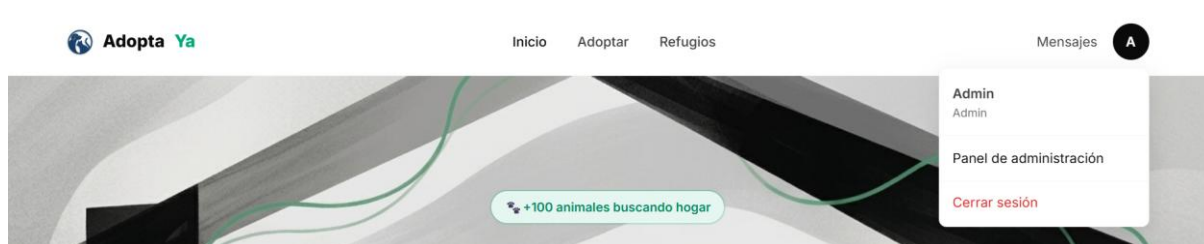


Ilustración 5 Menú de administrador



Ilustración 6 Menú de refugio

## Modal del formulario de adopción

El modal en vez de ser como una página diferente se superpone a la ficha de la mascota. Tiene un fondo casi transparente que cubre toda la pantalla. Dentro tiene un cuadro blanco con todo el formulario. El formulario contiene 9 campos donde el usuario debe poner los datos personales.



### Solicitud de adopción — Kiara

Cuéntanos un poco sobre ti para que el refugio pueda valorar tu solicitud.

Nombre completo \*

¿Eres mayor de edad? \*

☐

Sí, confirmo que soy mayor de edad

Dirección \*

Tipo de vivienda \*

Piso

¿Tienes jardín o terraza?

☐

Sí, dispongo de jardín o terraza

¿Tienes experiencia previa con mascotas?

☐

Ilustración 7 Modal de adopción

## Recorrido por las pantallas

A continuación, voy a mostrar las pantallas principales de la aplicación con una explicación.

### Página de inicio (Home)

Esta es la primera página que ve el usuario nada más entrar en la web. Está pensada para que nada más entres sepas de qué va la página web y que puede hacer. Está formada por:

- Un hero con una imagen principal y un botón de llamada a la acción.
- Una parte donde muestro estadísticas que aportan credibilidad.
- Una explicación de cómo funciona el proceso de adopción en cuatro pasos.
- Una vista previa de unos pocos refugios colaboradores

- Una llamada a la acción para que las protectoras se registren.

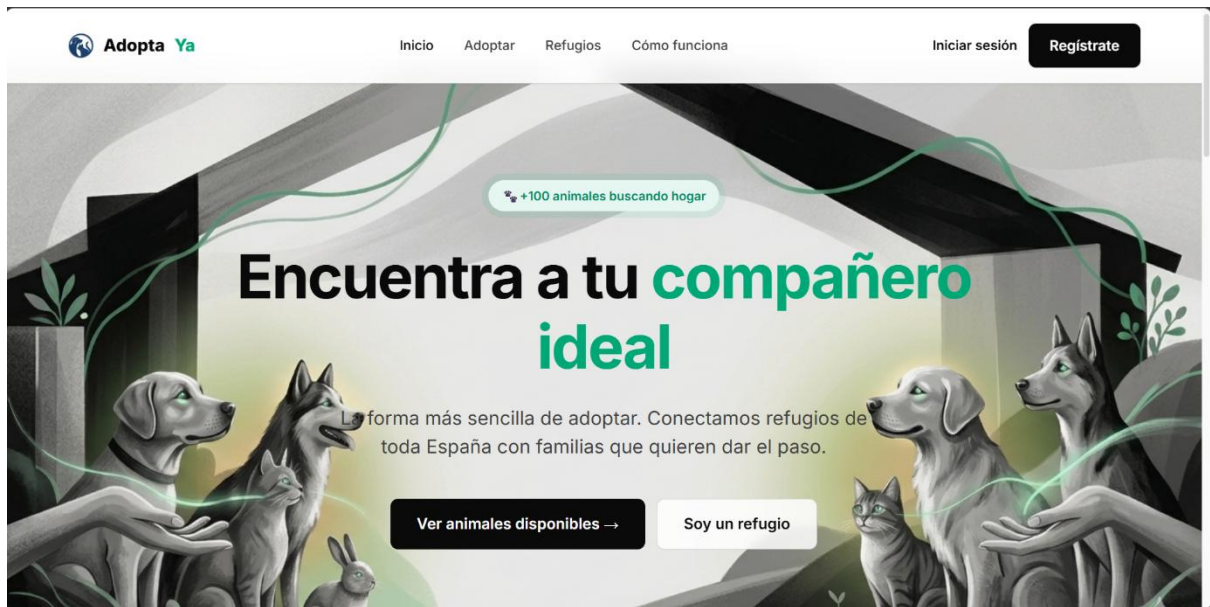


Ilustración 8 Pagina home

## Página de adopción (Adoptar)

Es donde el usuario puede ver todo el catálogo de mascotas. Tienes una barra de filtros donde puedes buscar directamente lo que quieres encontrar y al clicar sobre una mascota se abre una tarjeta de detalles.

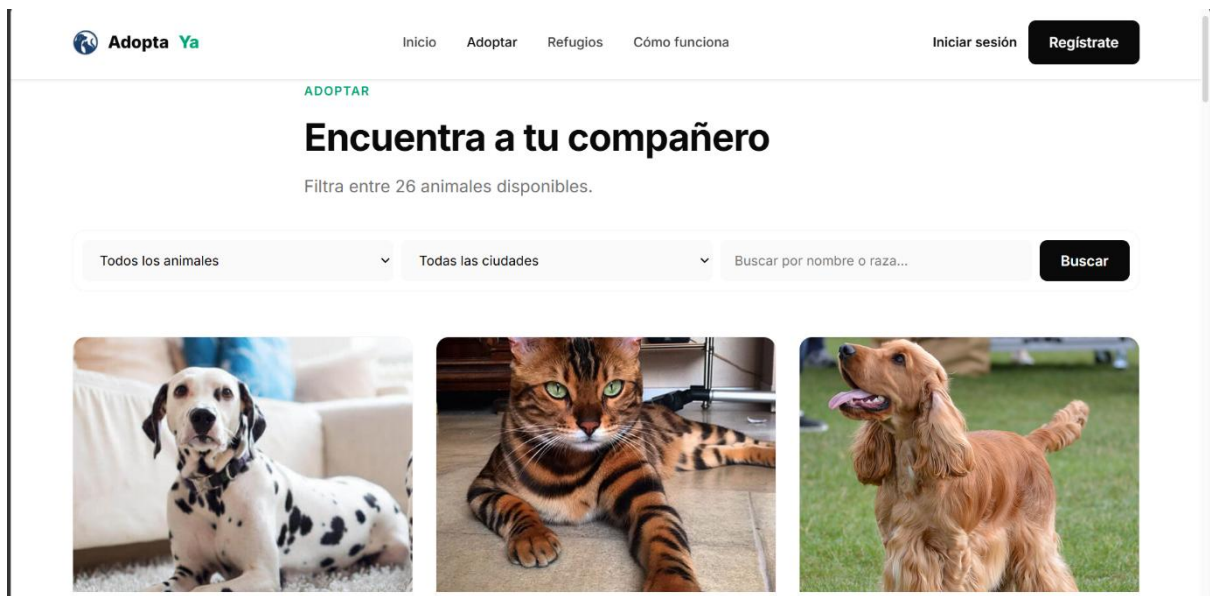


Ilustración 9 Pagina de adopción



## Detalle de la mascota

Cuando el usuario hace click en la mascota, se abre una tarjeta con toda la información de las mascotas:

- imagen grande de la mascota
- raza
- edad
- sexo
- ciudad
- descripción

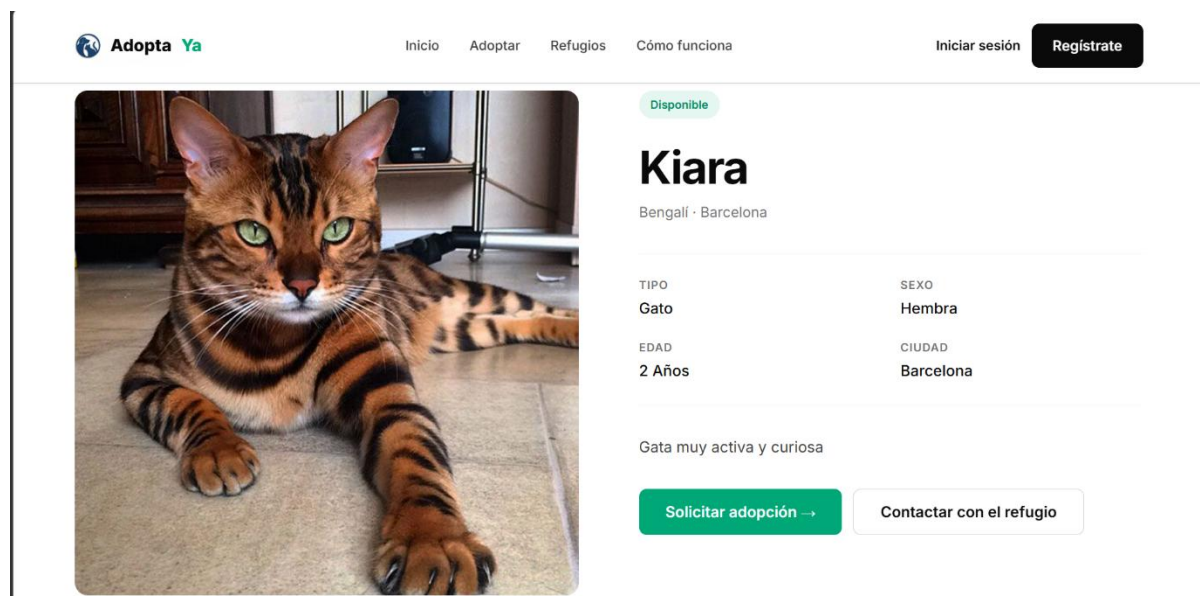


Ilustración 10 Pagina de detalle de la mascota

## Login y registro

El login y el registro están unidos por un toggle interno. Así si no tienes cuenta no tienes que ir a otra página sino únicamente darle al enlace que hay debajo del login y el formulario cambia al registro.

En el formulario de registro está la opción de tipo de cuenta (adoptante o refugio), lo que determina las funcionalidades a las que tendrá.

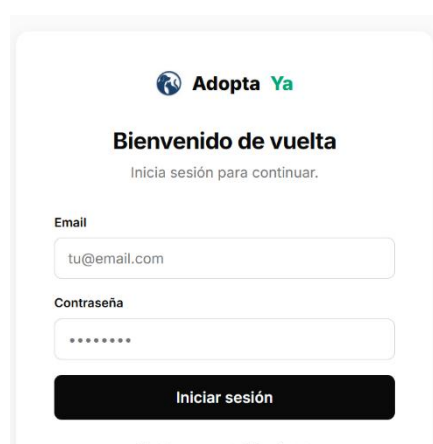


Ilustración 11 formulario de login

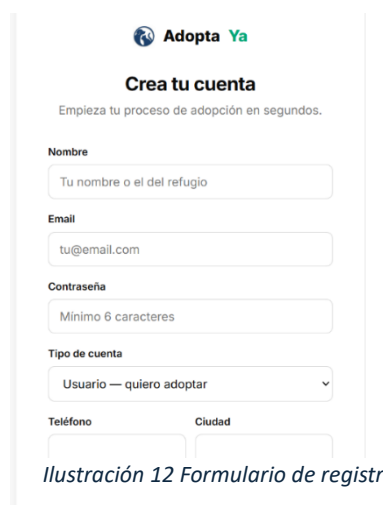


Ilustración 12 Formulario de registro

## Página de refugios

En esta página se ven todos los refugios que están registrados en la plataforma y puedes ver imagen, ciudad y número de mascotas que tienen publicadas. Al hacer click en uno se abre un modal con la información.

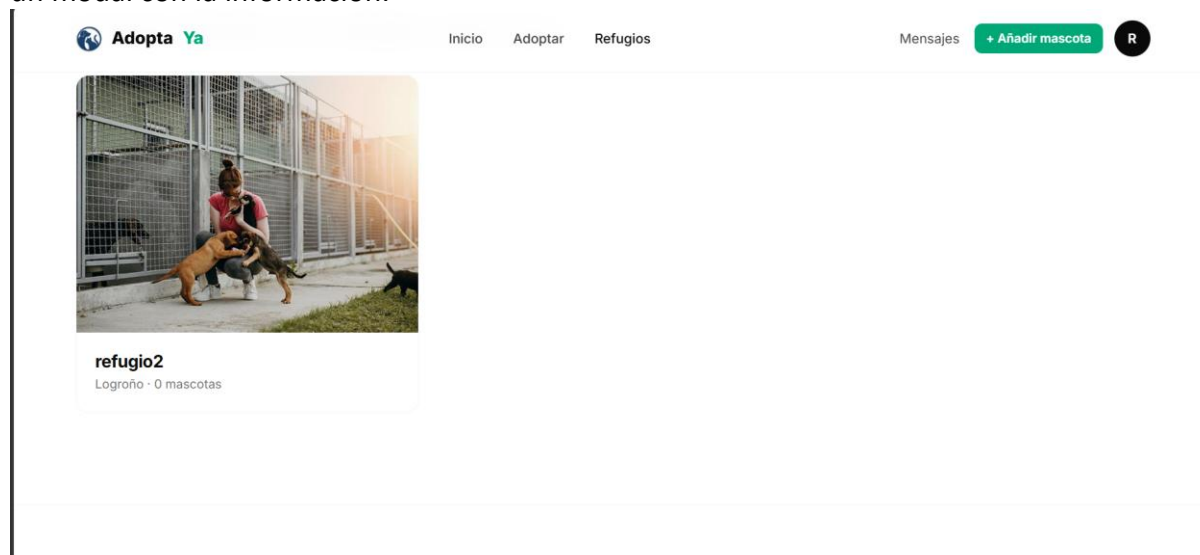


Ilustración 13 Pagina de refugios

## Mi refugio (formulario de protectora)

Esta es la pantalla donde un refugio se puede crear y editar su perfil que lo pueden ver los demás usuarios. La primera vez que entra ahí se ve un formulario en blanco. Y si ya tiene perfil salen los campos rellenos para poder editarlos.

## Mis mascotas

Un refugio cuando es verificado tiene, en el menú desplegable, dos opciones: “Mis mascotas” y “Solicitudes recibidas”. En la primera se ve un listado de todas las mascotas que ha publicado el refugio, donde puede cambiar el estado de la mascota. Y en la otra opción ve las solicitudes de adopción a sus mascotas, donde puede rechazarlas o aceptarlas.

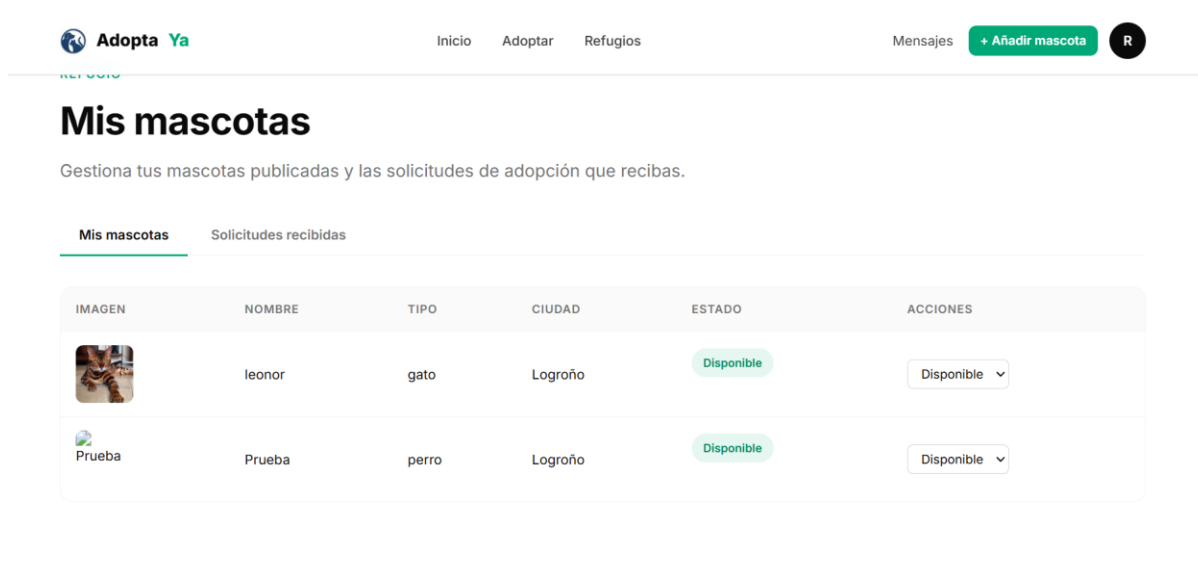


Ilustración 14 Pagina Mis mascotas

## Solicitudes recibidas (vista del refugio)

Principalmente esta página era simplemente una tabla, pero al añadir el formulario de las solicitudes de adopción los datos quedaban un poco ilegibles. Lo cambié por tarjetas como los refugios y las mascotas: son tarjetas con la información básica del adoptante, y al lado tiene un botón donde esa información se expande y sale toda la información que ha aportado previamente en el formulario. Y ahí puede cambiar el estado de la solicitud.

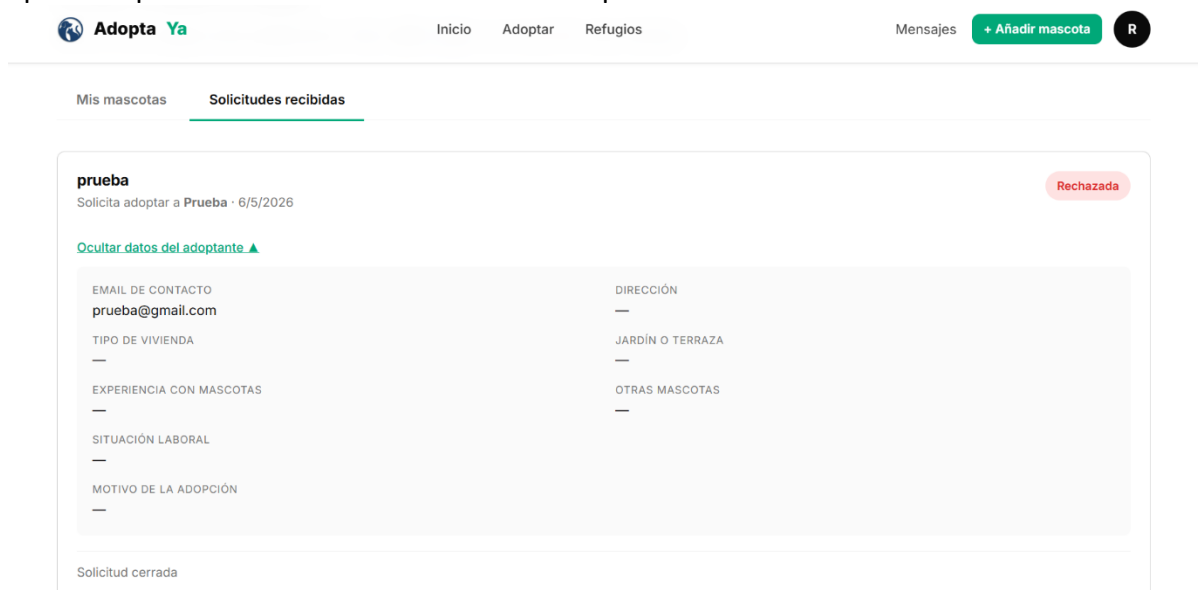


Ilustración 15 Solicitudes recibidas

## Mis solicitudes (vista del adoptante)

Un usuario para que no esté perdido de a cuantas mascotas ha solicitado adopción, he creado una pestaña que únicamente la pueden ver los adoptantes, donde es una lista de todos los animales a los que ha solicitado su adopción.

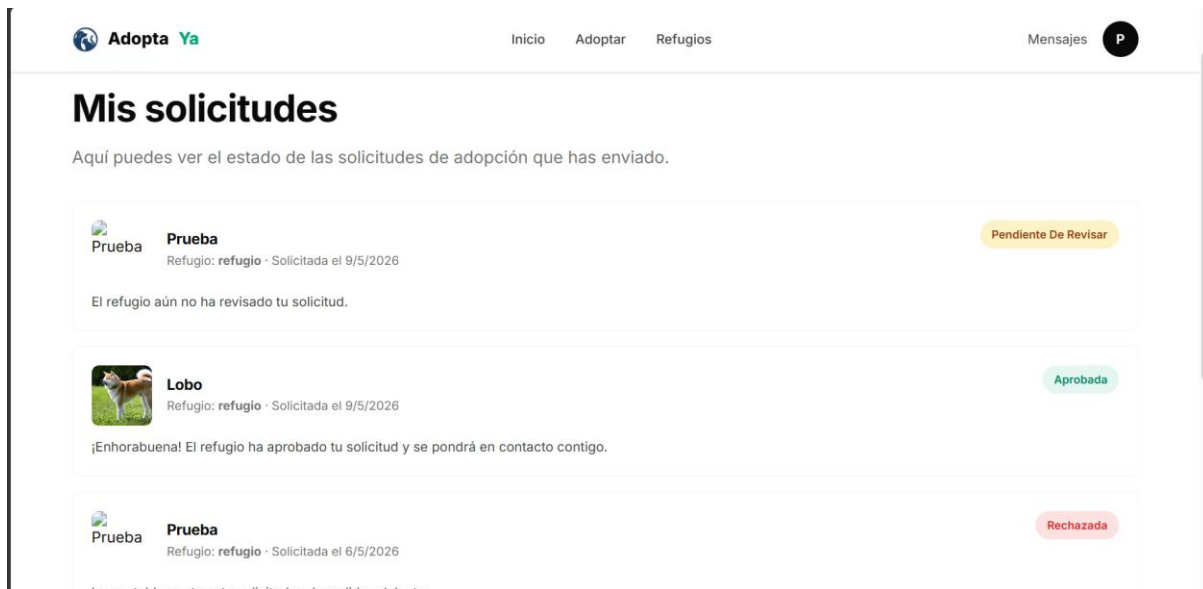


Ilustración 16 Mis solicitudes

## Añadir mascota (formulario de protectora)

Un formulario básico donde el refugio puede subir una mascota rellenando los datos y subiendo una foto. Una vez lo subes, la mascota aparece en el catálogo de todas las mascotas.

Ilustración 17 Formulario para añadir mascota

## Mensajes (chat interno)

La pantalla de mensajería tiene un diseño con dos columnas, donde a la izquierda tiene una columna con la lista de las conversaciones y a la derecha la conversación abierta, con los mensajes mandados. Los mensajes que manda el mismo usuario están a la derecha en verde y los recibidos a la izquierda en blanco.

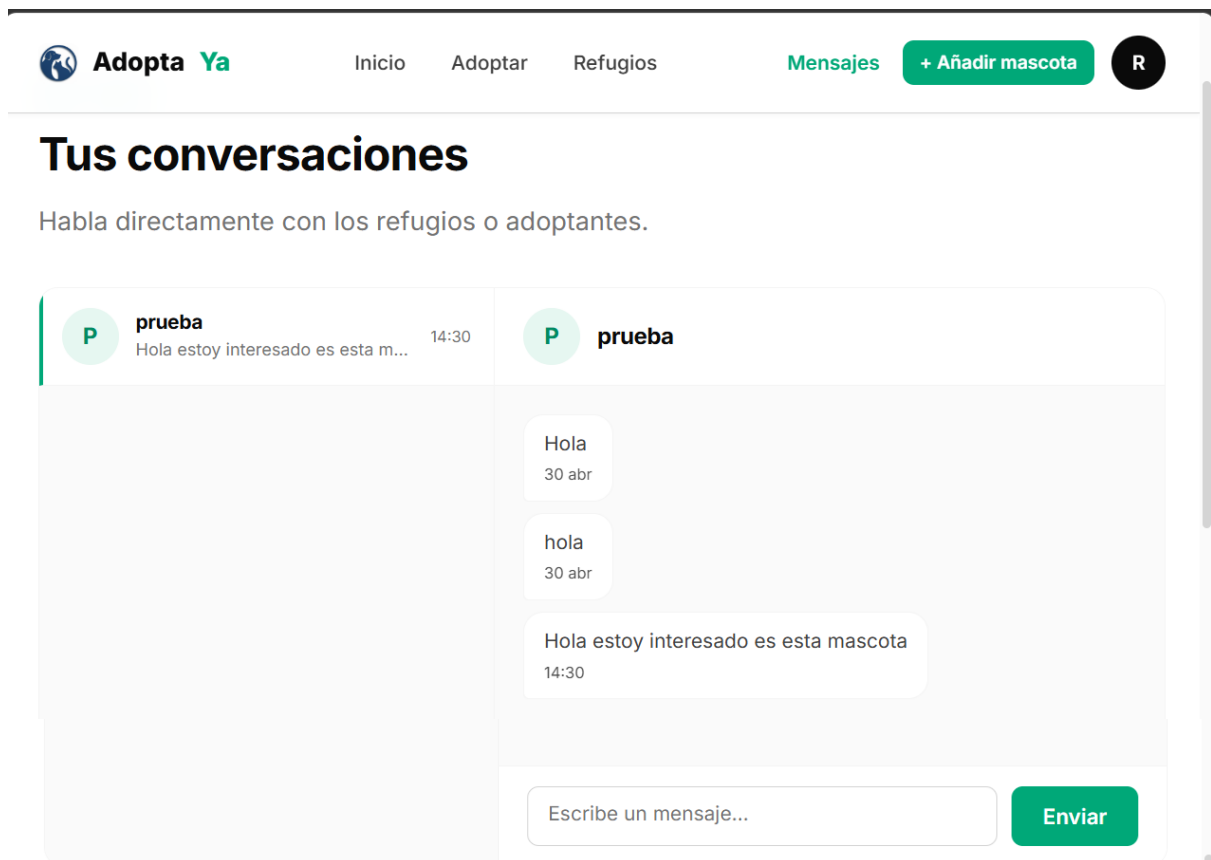


Ilustración 18 Pagina del chat

## Panel de administración

Y este panel solo está visible para los usuarios que sean administradores. Este panel sirve para gestionar las solicitudes, las mascotas y los usuarios. Puedes cambiar el rol a alguien, eliminar, aceptar o rechazar solicitudes.

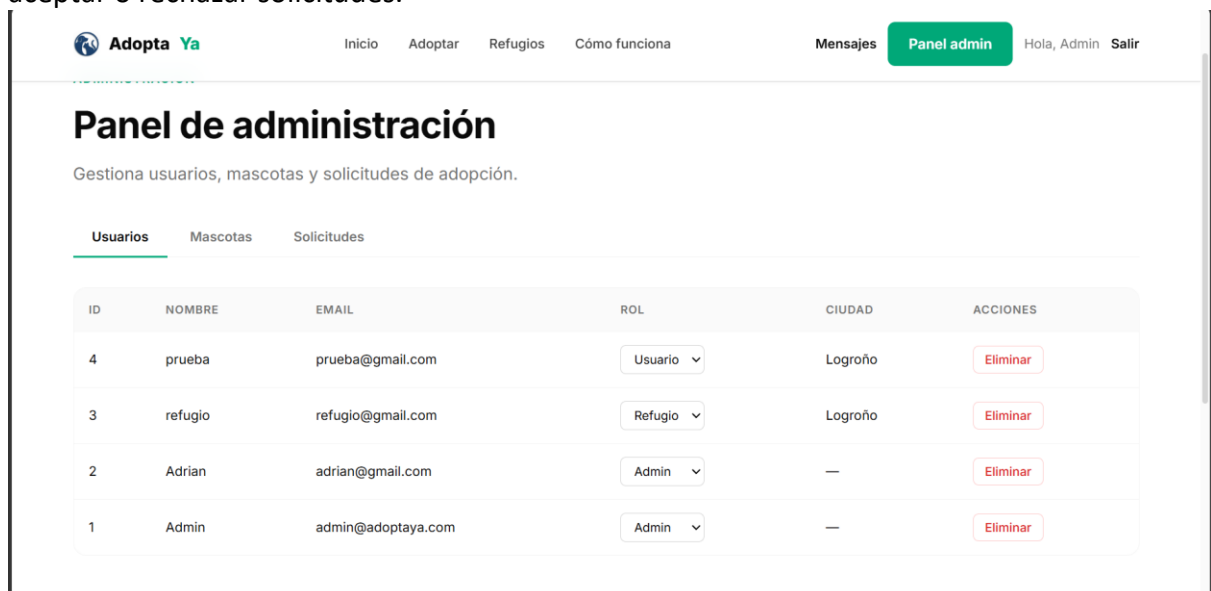


Ilustración 19 Panel de administrador

## 6. Implementación

Este apartado lo he dedicado para describir cómo se ha desarrollado mi proyecto, organizando el proceso por fases y explicando qué decisiones he tomado en cada apartado.

### 6.1 Tecnologías empleadas

El proyecto se ha desarrollado con JavaScript: React + Vite en el frontend, con Node.js + Express + MySQL en el backend, y JWT, bcrypt, Muller y CORS como librerías de apoyo. La comunicación entre frontend y backend se hace con una API REST que los datos que intercambia lo hacen en formato JSON.

### 6.2 Desarrollo de la aplicación (fases)

El desarrollo lo he dividido en 5 fases. En el plan original eran seguidas, pero en la práctica alguna fase se va solapando, sobre todo el frontend y el backend.

#### Fase 1: Diseño de las bases de datos

Lo primero que hice fue la estructura de la base de datos, ya que el modelo de datos condiciona todo lo demás. Definí las cinco tablas, los campos de cada una y las relaciones entre ellas.

#### Fase 2: Construcción del backend

El backend lo he organizado con una estructura por capas para mantener cada cosa en su sitio y no liarme. Que esté organizado en capas hace que cada archivo tenga una responsabilidad: las rutas indican que URL responde a qué función, los controladores tienen toda la lógica, y los middlewares son los encargados de las tareas de autenticación.

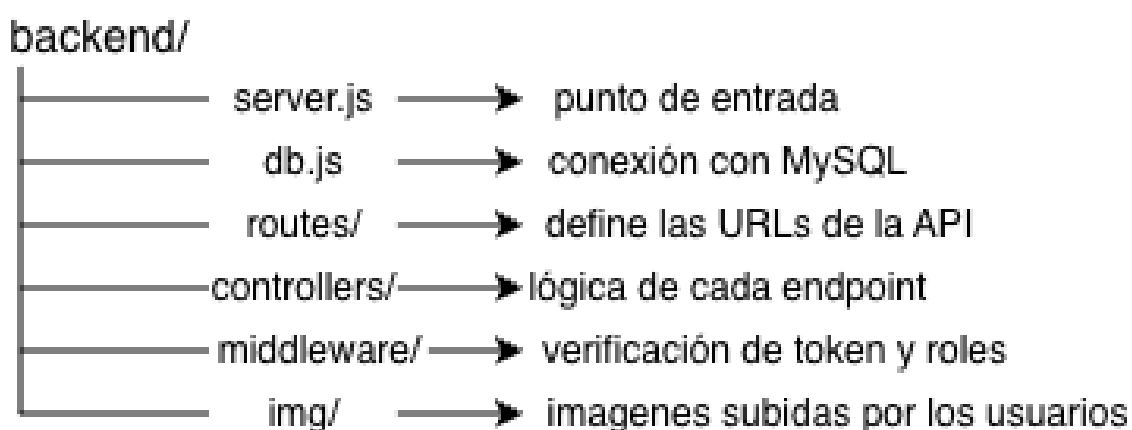


Ilustración 20 Arquitectura del backend

- **Autenticación con JWT:** cuando un usuario inicia sesión, se genera un token con la información de su id y rol de forma cifrada, y se envía al frontend. A partir de ahí, el frontend incluye ese token en todos lados que la petición necesite estar logueado. El middleware `verifyToken` es el encargado de verificar ese token para poder seguir con las peticiones del usuario.
- **Hasheo de contraseñas:** las contraseñas, por tema de seguridad, nunca se guardan en forma de texto plano. Cuando un usuario se registra, su contraseña se hashea antes

de guardarse en la base de datos. Y al hacer login esa contraseña se hashea también y se compara con la que está guardada en la base de datos.

- **Subida de imágenes con Multer:** las imágenes de las mascotas y las de los perfiles de los refugios se suben al servidor con multipart/form-data. Multer recibe la imagen, le asigna un nombre que se basa en la fecha y la guarda en la carpeta de las imágenes del backend.

### Fase 3: Construcción del frontend

El frontend lo hice con React, y está organizado en componentes y páginas.

#### Front-end/src/

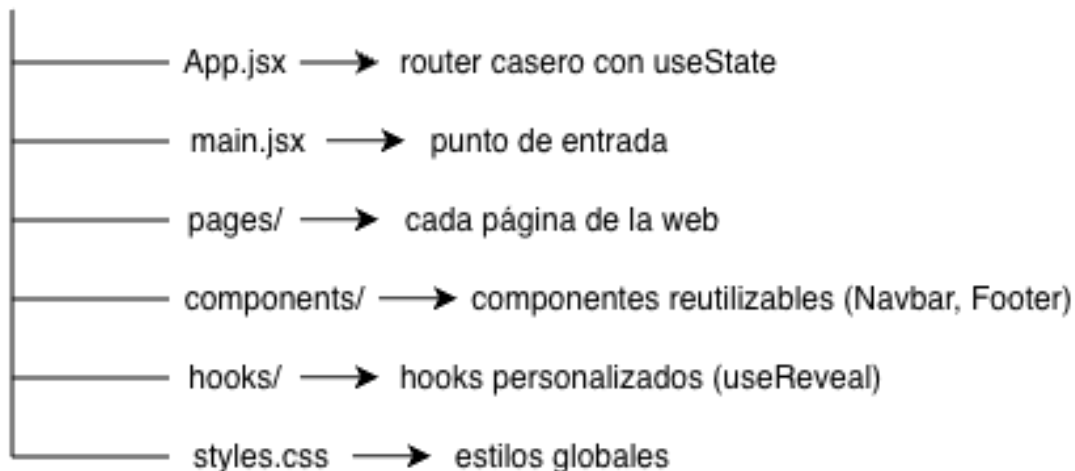


Ilustración 21 Arquitectura de frontend

- **Sistema de rutas:** en vez de usar React Router, opté por implementar un sistema de rutas propio basado en useState y el hash de la URL. El archivo App.jsx es el encargado de leer el hash, decidir qué página hay que mostrar y proporciona una función “navegar()” que es la que las demás páginas utilizan para poder cambiar de ruta.
- **Gestión del estado del usuario:** cuando el usuario inicia sesión, sus datos y el token del que hemos hablado antes se guardan en el localStorage. Así la sesión es persistente y tengas la cuenta iniciada, pero solo por 7 días que es lo que aguanta el token, después se renueva al iniciar sesión.
- **Comunicación con el backend:** las páginas que necesitas datos del backend, hacen peticiones fetch. Las que necesitan la autenticación, tienen el token en la cabecera.

### Fase 4: Funcionalidades avanzadas

Una vez tengo la base de la página web, he ido añadiendo las funcionalidades más complejas:

- **Sistema de roles:** añadí un middleware en el que compruebo el campo “tipo” de la tabla “usuario” una vez tengo el rol, le asigno funciones diferentes para cada uno de ellos.
- **Panel de administración:** esta es una página exclusiva para los usuarios que tienen como rol “administrador”. Tiene tres pestañas (usuarios, mascotas y solicitudes). Donde puedes gestionar todo.

- **Sistema de mensajería:** lo he implementado con polling. Donde el frontend cada 5 segundos le pregunta al backend que si hay mensajes nuevos. en una conversación ya abierta, y cada 30 segundos comprueba que si hay mensajes nuevos en general.
- **Sistema de verificación de refugios:** amplié el formulario de registro si en la opción de tipo de usuario es la de refugio, esa opción es la de CIF, por la cual el administrador puede verificar que es una protectora de verdad y en ese caso aceptar la cuenta y el usuario podría realizar las funcionalidades de un refugio
- **Sistema de adopciones avanzadas:** este es el proceso más complejo de la aplicación, ya que combina formulaciones, validaciones y control de estados para hacer que sea un proceso realista. Tiene un formulario que se abre cuando el usuario le da al botón de solicitar adopción, donde pide datos personales para que el refugio conozca un poco más a la persona que quiere adoptar a sus mascotas. Luego la mascota va pasando por diferentes estados.

## Fase 5: Pulido del diseño y pruebas

Esta es la última fase, aquí refiné el diseño y probé a fondo la aplicación. Para el diseño hice lo siguiente:

- Elegí una paleta de colores básica de tres colores y una sola tipografía.
- Puse interacciones sutiles: hovers en tarjetas, una pequeña animación al hacer scroll, transiciones suaves en los botones.
- Aseguré que sería responsive mediante media queries.

## 6.3 Despliegue

Para poder ejecutar la aplicación en cualquier ordenador sin tener que instalar Node.js, MySQL o configurar el entorno a mano, decidí utilizar Docker y Docker compose para desplegar mi proyecto. Esto permite que todos los archivos o estructura del proyecto se levante con un único comando: Docker compose up.

Antes de desplegar mi aplicación, valoré las opciones que había para desplegarla. Pensé en un despliegue sencillo en local o un despliegue público en la nube con Vercel y Render. Al final decidí hacerlo con Docker, así me aseguro de tener portabilidad total de la página.

### Arquitectura

Se ha dividido en tres contenedores levantados con Docker Compose:

| Contenedor        | Imagen base    | Función                  |
|-------------------|----------------|--------------------------|
| Adoptaya-mysql    | Mysql:8.0      | Bases de datos           |
| Adoptaya-backend  | Node:20-alpine | API REST de Express      |
| Adoptaya-frontend | Nginx:alpine   | Frontend React compilado |

Solo el contenedor del frontend tiene un puerto hacia el exterior, así la aplicación es accesible desde una única URL. Los demás contenedores están aislados y se comunican entre ellos a través de red interna del propio Docker, así mejora la seguridad.



Nginx, no solo sirve el frontend, hace también de proxy inverso hacia el backend. Así, el navegador solo ve una URL y se evitan problemas con el CORS.

## Adaptaciones en el código

Para no tener que hacer muchos cambios, y que mi código funcionase tanto en local como dentro de Docker, únicamente hice dos cambios en el backend. Primero, la conexión a la base de datos lee variables de entorno en vez de tener los valores hardcodeados, con valores por defecto para el modo local.

Segundo, sustituí `mysql.createConnection()` por `mysql.createPool()`. Este cambio es porque cuando el backend se arranca dentro del Docker, MySQL podría estar iniciándose y la primera conexión podía fallar.

El frontend usa un multi-stage build: esto se separa en dos fases, en la primera Node.js compila el código, y en la segunda Nginx solo copia la carpeta `dist/`.

## Persistencia de datos

Los datos de MySQL se guardan en un volumen persistente llamado `mysql-data`, eso hace que cuando apague y encienda el sistema, los datos sigan y no se borren. Las imágenes subidas por un usuario también se montan como un volumen, para que queden persistentes y no se pierdan.

## Problemas encontrados

Tuve que resolver algún problema mientras desplegaba la aplicación: el backend al principio perdía la conexión con MySQL porque arrancaba antes que la BBDD y lo pude solucionar con `createPool`, Nginx confundía las imágenes del frontend como el hero y el logo con las del backend y al principio no se mostraban, y también tuve que cambiar las URLs del frontend para construirlas mediante variables de entorno y para que funcione sin cambios en todos los entornos.

# 7. Pruebas

## 7.1. Pruebas funcionales

Lo primero que hice al terminar el proyecto, fue comprobar todas las funcionalidades principales. Y así me aseguré de que todo funcionaba bien.

Resultados generales

| Caso de prueba                                | Resultado                |
|-----------------------------------------------|--------------------------|
| Registro de un adoptante                      | Correcto                 |
| Registro de un refugio o protectora           | Correcto                 |
| Inicio de sesión con una cuenta valida        | Correcto                 |
| Inicio de sesión con una cuenta sin registrar | Sale un mensaje de error |
| Listado y el filtrado de mascotas             | Correcto                 |

|                                                                                               |                                               |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------|
| Tarjeta de detalle de una mascota                                                             | Correcto                                      |
| Solicitud de adopción                                                                         | Correcto                                      |
| Añadir una mascota                                                                            | Correcto                                      |
| Subir una imagen                                                                              | Correcto                                      |
| Editar el perfil de refugio                                                                   | Correcto                                      |
| Ver la página publica de un refugio                                                           | Correcto                                      |
| Enviar y recibir mensajes                                                                     | Correcto                                      |
| Aceptar una solicitud siendo admin                                                            | La mascota aparece como “adoptada”            |
| Eliminar usuario desde el panel de administración                                             | Correcto                                      |
| Adoptante envía solicitud de adopción sin marcar la casilla de mayor de edad.                 | Muestra un error y no se envía.               |
| Adoptante intenta enviar solicitud dos veces a una misma mascota                              | Se rechaza la segunda solicitud               |
| Refugio acepta una solicitud de una mascota donde tenia mas de una solicitud                  | Las demás solicitudes pasan a ser rechazadas. |
| Refugio intenta marcar como adoptada una mascota que está como disponible y no como reservada | Sale un código 400                            |

## Caso destacado – Flujo de adopción

En mi proyecto una de las cosas más importantes que hay es el flujo de adopción. Aquí hay dos opciones de aceptar una solicitud.

### Pasos que seguí:

1. Inicié sesión con una cuenta con el rol de adoptante.
2. Entré a la lista de las mascotas y seleccioné una anteriormente creada por un refugio.
3. En la tarjeta de detalle de la mascota seleccioné el botón de solicitar adopción.
4. Ahora inicié sesión con una cuenta de administrador y desde el panel de administración acepté la solicitud.
5. Luego solicité otra adopción.
6. Entré con una cuenta de refugio y desde ahí acepté la solicitud.
7. Las mascotas pasan de disponibles a adoptadas

## Caso destacado — Flujo completo de una adopción

Para probar el flujo completo de una adopción seguí los siguientes pasos:

1. Inicié sesión con un usuario llamado prueba1 y solicité la adopción de una mascota, rellenando el formulario y todo.
2. Inicié sesión con otro usuario de prueba diferente y solicité adopción del mismo animal.
3. Las dos solicitudes pasaron a estar en pendiente en la base de datos.

4. Entré con la cuenta del refugio dueño de esa mascota. Entre a la pestaña de Mis mascotas -> Solicitudes recibidas. Vi las dos solicitudes y pude ver toda la información del formulario.
5. Pulsé “Marcar en revisión” en la solicitud de prueba1. La tarjeta cambió de estado.
6. Después le di a “Aprobar” en la misma solicitud. Pasaron 3 cosas:
  - a. La solicitud de prueba1 pasó a “aprobada”.
  - b. La solicitud de prueba2 pasó a ser rechazada.
  - c. La mascota pasó a estar “reservada”, hasta que el refugio le dé a “adoptada” cuando la entregue.
7. Verifiqué que la mascota ya no aparecía como disponible en el catálogo.
8. Como refugio en la pestaña de mis mascotas, me salía la mascota que había aceptado a prueba1 y me salía la opción de “Marcar como adoptada”. Lo pulsé y pasó a estar en estado de “adoptado”.

## Caso destacado – Sistemas de mensajería

Esta prueba la hice con dos navegadores abiertos a la vez, así me permitía ver al momento si los mensajes llegaban bien.

### Pasos que seguí

1. En el primer navegador, inicié sesión con una cuenta de adoptante, entré a una mascota y le di al botón “Contactar con el refugio”.
2. Escribí un mensaje y le di a enviar.
3. En el otro navegador inicié sesión como refugio, al entrar vi q en el botón de mensajes ponía que había 1.
4. Entré y vi que tenía un mensaje del otro usuario y respondí
5. Volví al otro usuario y vi que el refugio había respondido.

## 7.2. Pruebas no funcionales

| Caso de prueba                                                                            | Resultado                                                              |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Intento de acceder a una ruta privada sin el token                                        | La petición es rechazada                                               |
| Refugio sin verificar intenta publicar una mascota                                        | El middleware rechaza la petición                                      |
| Otro refugio intenta cambiar el estado de una solicitud que no es de una de sus mascotas. | El backend lo rechaza con 403                                          |
| Inicio de sesión con una cuenta sin registrar                                             | La interfaz se adapta correctamente sin desbordes ni scroll horizontal |
| Reiniciar los contenedores de Docker                                                      | Los datos se guardan gracias al volumen mysql-data                     |
| Tiempo de respuesta de las peticiones mas usadas                                          | Por debajo de 200 ms en condiciones normales                           |

|                             |          |
|-----------------------------|----------|
| Solicitud de adopción       | Correcto |
| Añadir una mascota          | Correcto |
| Subir una imagen            | Correcto |
| Editar el perfil de refugio | Correcto |

### 7.3. Pruebas de seguridad

| Caso de prueba                                                                    | Resultado                               |
|-----------------------------------------------------------------------------------|-----------------------------------------|
| Un refugio sin verificar intenta iniciar sesión                                   | Devuelve un fallo 403 con un mensaje    |
| El admin rechaza la verificación a un refugio y este intenta hacer alguna función | El middleware lo bloquea en la petición |

#### Caso destacado — Flujo de verificación de un refugio

Para probar el ciclo, seguí los siguientes pasos:

1. Primero me cree una cuenta de un refugio con un CIF inventado
2. Al crear la cuenta me salió el mensaje de “Solicitud recibida”.
3. Sin la verificación de cuenta, intenté iniciar sesión y me salió el mensaje de “Cuenta pendiente de verificación”.
4. Inicié sesión como administrador, entre al panel de administrador en la pestaña de “Usuarios”, y de primeras el usuario nuevo sale con el badge amarillo de “pendiente” y su CIF al lado.
5. Verifiqué el usuario y el badge de amarillo pasó a verde.
6. Inicié sesión con el usuario de refugio y ya podía hacer todas las funcionalidades.

El flujo funciona correctamente y como yo esperaba. La verificación garantiza que ningún usuario sin verificar no pueda añadir mascotas ni hacer nada que pueda hacer un refugio verificado.

## 8. Conclusiones

### 8.1. Problemas encontrados

En el proceso de realizar mi proyecto me he encontrado con varios obstáculos que han hecho que avance más lento en algunos momentos.

#### Orden de los middlewares en Express

Uno de los errores más confusos que he tenido ha sido el orden de las rutas que registraba en el servidor. En uno de los middlewares pedía token para todas las peticiones que hacía, entonces las rutas que iban después se quedaban sin acceso. Tardé un rato en darme cuenta de que el problema era el orden en el que estaban registradas las rutas en server.js.

## Animaciones que ocultaban el contenido

Añadí unas animaciones que aparecen al hacer scroll, al principio me di cuenta de que solo aparecían en la página principal y no en la de adopción. Esto pasaba porque el observer se cargaba antes que las mascotas del backend. Lo solucioné haciendo que el hook useReveal aceptara una dependencia, de manera que se reinicia cuando los datos llegan.

## Configuración de Git y .gitignore

La primera vez que fui a hacer git push con todo montado, con el servidor, etc. No me dejaba subirlo y al rato me di cuenta de que era por los node\_modules y pensé en hacer un archivo .gitignore.

## 8.2. Futuras mejoras

Mi proyecto es una aplicación funcional y completa para los objetivos que me puse para la aplicación, pero sigue habiendo más cosas que se puedan mejorar y añadir. Algunas de las ideas de una futura mejora son las siguientes:

- **Notificación por email:** aunque el chat dentro de la aplicación es útil y cubre la comunicación entre adoptante y refugio, sería útil e interesante que cuando te manden un mensaje en la aplicación te llegue una notificación al email.
- **Galería de imágenes:** en el catálogo de las imágenes ahora mismo solo tienen una imagen, pero estaría bien que tuvieran varias imágenes en la misma tarjeta.
- **Mapa de refugios:** añadir un mapa donde muestra la ubicación de cada refugio.
- **Perfil de usuario editable:** estaría bien que los adoptantes también pudieran editar sus datos.
- **Verificación de refugios:**
- **Mascota interactiva:** pensé en poner una mascota interactiva en la pantalla en la página de mascotas y poder interactuar de alguna manera o que te siga con la mirada el ratón, etc.
- **Verificación de refugios automática:** sería interesante implementar una API externa que verifique automáticamente si el CIF del registro existe de verdad o no y que el usuario del refugio se verifique sin necesidad de que el administrador verifique la cuenta.
- **Notificación por email de la verificación del refugio:** que el usuario con el rol de refugio reciba un email cuando se verifique su cuenta.
- **Notificaciones por email al adoptante:** que le llegue una notificación de email al adoptante cuando el estado de la mascota que haya solicitado cambie.

## 8.3. Resumen final

Cuando me asignaron el tema de TFG, al principio no tenía claro cómo empezar a realizarlo ni como enfocarlo. Pero empecé a investigar un poco sobre el mundo de la adopción de mascotas y de cómo realizar una página sobre eso y qué funciones son necesarias que la aplicación haga para poder completar el proceso de adopción de la forma más fácil y cómoda posible. Ahí me di cuenta de que era un ámbito en el que la tecnología puede marcar una

diferencia, ya que, gracias a mi proyecto, muchos animales pueden tener un nuevo y mejor hogar y es eso lo que me motivó para tomármelo en serio.

Durante el desarrollo de este proyecto me he tenido que enfrentar a muchas decisiones importantes para el buen planteamiento de la página: cómo estructurar bien la base de datos, que tecnologías combinen entre ellas, cómo proteger las contraseñas, cómo diseñar una interfaz que se vea bien, etc. Todo esto me ha hecho salir de mi zona de confort e investigar tecnologías nuevas y más comunes en el mundo laboral. Y esto me ha ayudado a familiarizarme más con las tecnologías más utilizadas actualmente.

Estoy muy satisfecho del resultado ya que no es un proyecto que destaque mucho por su complejidad, pero sí que cumple con su función y tiene todo lo que debe tener una aplicación de este tipo, está bien organizada y es muy intuitiva para los usuarios que sean nuevos. Para mí que una aplicación sea intuitiva es la mejor forma para medir que una aplicación sea buena y esté bien hecha.

## 9. Bibliografía

Para realizar este proyecto al principio estaba un poco verde y no estaba familiarizado con las tecnologías que he utilizado. Por eso he tenido que investigar para poder sacar el proyecto adelante. Y visité estos distintos lugares:

### Documentación oficial

- React -> <https://react.dev/>
- Vite -> <https://vite.dev/>
- Node.js -> <https://nodejs.org/docs/>
- Express -> <https://expressjs.com/>
- MySQL -> <https://dev.mysql.com/doc/>
- Referencias para HTML, CSS y JavaScript -> <https://www.w3schools.com/>

### Librerías utilizadas

- mysql2 -> <https://www.npmjs.com/package/mysql2>
- jsonwebtoken -> <https://www.npmjs.com/package/jsonwebtoken>
- bcryptjs -> <https://www.npmjs.com/package/bcryptjs>
- multer -> <https://www.npmjs.com/package/multer>
- cors -> <https://www.npmjs.com/package/cors>

## Herramientas utilizadas

- GitHub -> <https://github.com/>
- Visual Studio Code -> <https://code.visualstudio.com/>
- Dbdiagram.io -> <https://dbdiagram.io/>
- Google Fonts -> <https://fonts.google.com/>

## Web analizadas

- Wakyma -> <https://www.wakyma.com/>
- Barkibu -> <https://www.barkibu.com/es/>